Bibliothek M295/294

Projektdokumentation

Full-Stack Buchverwaltungssystem

Autor	Indujan Vijayarajah
Modul	M295 – Backend realisieren
Datum	März 2026
Technologien	Spring Boot / React / H2

1. Einführung

Das Modul M295 verfolgt das Ziel, ein vollständiges Webprojekt mit Backend und Frontend zu realisieren. Im Rahmen dieser Leistungsbeurteilung wurde eine Bibliothek-Verwaltungsapplikation entwickelt, die es einem Benutzer ermöglicht, Bücher zu verwalten – inklusive Authentifizierung, CRUD-Operationen und weiteren Zusatzfunktionen.

1.1 Ziel

Die Applikation soll folgende Kernfunktionalitäten bieten:

- Vollständige CRUD-Operationen für Bücher (Erstellen, Lesen, Aktualisieren, Löschen)
- JWT-basierte Benutzerauthentifizierung (Registrierung und Login)
- Suchfunktion, Kategoriefilter und Sortierung
- Export der Buchliste als PDF und CSV/Excel
- Dunkles und helles Design (Dark/Light Mode)
- Mehrsprachigkeit (Deutsch / Englisch)
- Abgabedatum und Erinnerungssystem für Bücher
- Einstellungsseite mit Profilverwaltung

1.2 Rahmenbedingungen

Die Applikation wurde lokal entwickelt und läuft auf folgenden Ports:

- Backend: http://localhost:8080
- Frontend: http://localhost:5173

2. Architektur & Technologien

2.1 Verwendete Technologien

Feature	Beschreibung
Spring Boot 3	Backend-Framework für REST-API und Sicherheit
Spring Security	JWT-Authentifizierung und Autorisierung
H2 Database	In-Memory-Datenbank (läuft im RAM, kein Setup nötig)
React 18	Frontend-Framework für Single Page Application
Vite	Build-Tool und Entwicklungsserver für React
jsPDF (CDN)	PDF-Export direkt im Browser ohne Installation

2.2 Projektstruktur Backend

```
LibraryM295-new/
  src/main/java/ch/library/
    controller/   → BookController.java, AuthController.java
    model/        → Book.java, User.java
    repository/   → BookRepository.java, UserRepository.java
    security/     → JwtUtil.java, JwtFilter.java, SecurityConfig.java
    service/      → (Business Logic)
  src/main/resources/
    application.properties
```

2.3 Projektstruktur Frontend

```
library-frontend/
  src/
    App.jsx       → Gesamte Applikation (React)
    main.jsx      → Einstiegspunkt
  index.html
  package.json
  vite.config.js
```

3. Backend – Spring Boot

3.1 REST-Endpunkte

Methode	Endpunkt	Auth	Beschreibung
POST	/auth/register	Nein	Neuen Benutzer registrieren
POST	/auth/login	Nein	Login – gibt JWT Token zurück
POST	/auth/change-password	Ja	Passwort ändern
DELETE	/auth/delete	Ja	Konto löschen
GET	/books	Ja	Alle Bücher abrufen
POST	/books	Ja	Neues Buch erstellen
PUT	/books	Ja	Buch aktualisieren
DELETE	/books/{id}	Ja	Buch löschen

3.2 Datenmodell – Book

```
@Entity
public class Book {
    @Id @GeneratedValue
    private Long id;
    private String title;
    private String author;
    private String category;
    @Column(length = 1000)
    private String notes;
    private LocalDate dueDate;
}
```

3.3 JWT Authentifizierung

Die Sicherheit der API wird durch JWT (JSON Web Tokens) gewährleistet:

- Bei Login/Registrierung wird ein Token generiert (Gültigkeit: 24 Stunden)
- Jede geschützte Anfrage muss den Token im Header mitsenden: Authorization: Bearer {token}
- Der JwtFilter validiert den Token bei jeder Anfrage automatisch
- Bei ungültigem Token wird HTTP 401 zurückgegeben

4. Frontend – React

4.1 Seiten & Navigation

Die Applikation besteht aus folgenden Seiten, erreichbar über die Navigationsleiste:

Feature	Beschreibung
📚 **Bücher**	Hauptseite: Bücher hinzufügen, bearbeiten, löschen, suchen und filtern
🔔 **Erinnerungen**	Alle Bücher mit gesetztem Abgabedatum, sortiert nach Fälligkeit
⚙️ **Einstellungen**	Passwort ändern, Kontaktdaten, Design, Konto löschen

4.2 Funktionen im Überblick

Feature	Beschreibung
CRUD	Bücher erstellen, lesen, bearbeiten und löschen
Suche	Echtzeit-Suche nach Titel und Autor
Kategoriefilter	Filtern nach Roman, Sachbuch, Krimi, Fantasy usw.
Sortierung	A–Z, Z–A, Neueste zuerst, Favoriten zuerst
Favoriten ⭐	Bücher als Favoriten markieren
Gelesen/Ungelesen	Lesestatus pro Buch setzen
Notizen	Eigene Notizen zu jedem Buch hinzufügen
Abgabedatum	Rückgabetermin setzen mit farbiger Status-Anzeige
Abgegeben-Haken	Bücher als abgegeben markieren (auch im PDF sichtbar)
PDF Export	Buchliste als PDF herunterladen (mit Checkboxen)
Excel Export	Buchliste als CSV-Datei herunterladen
Dark/Light Mode	Umschalten zwischen dunklem und hellem Design
Mehrsprachig	Deutsch und Englisch vollständig unterstützt
Tutorial	Interaktives Onboarding beim ersten Login

4.3 Statistik-Anzeige

Oben auf der Bücher-Seite werden folgende Zahlen live angezeigt:

- Bücher total – Gesamtanzahl aller Bücher
- Autoren – Anzahl einzigartiger Autoren
- Kategorien – Anzahl verwendeter Kategorien
- Gelesen – Anzahl als gelesen markierter Bücher
- Ungelesen – Verbleibende ungelesene Bücher

- Favoriten – Anzahl mit Stern markierter Bücher

5. Installation & Inbetriebnahme

5.1 Voraussetzungen

- Java 17 oder höher
- Node.js 18 oder höher
- Maven (im Backend bereits integriert als mvnw)

5.2 Backend starten

```
# In den Backend-Ordner wechseln
cd LibraryM295-new

# Server starten
./mvnw spring-boot:run

# Windows:
mvnw.cmd spring-boot:run

# Server läuft auf: http://localhost:8080
```

5.3 Frontend starten

```
# In den Frontend-Ordner wechseln
cd library-frontend

# Abhängigkeiten installieren (nur beim ersten Mal)
npm install

# Entwicklungsserver starten
npm run dev

# App öffnen: http://localhost:5173
```

5.4 Erste Schritte

1. http://localhost:5173 im Browser öffnen
2. Tab "Registrieren" wählen und Konto erstellen
3. Tutorial folgen oder direkt Bücher hinzufügen
4. Titel, Autor, Kategorie eingeben und auf "+Hinzufügen" klicken

6. Testing

6.1 Manuelle Tests

Testfall	Erwartetes Ergebnis	Status
Registrierung mit neuem Benutzer	Weiterleitung zur App, Token gespeichert	☑ OK
Login mit falschem Passwort	Fehlermeldung erscheint	☑ OK
Buch hinzufügen	Buch erscheint in der Liste	☑ OK
Buch bearbeiten	Änderungen werden gespeichert	☑ OK
Buch löschen	Buch wird aus Liste entfernt	☑ OK
Suche nach Titel	Nur passende Bücher angezeigt	☑ OK
PDF Export	buecher.pdf wird heruntergeladen	☑ OK
Excel Export	buecher.csv wird heruntergeladen	☑ OK
Abgabedatum setzen	Status-Badge erscheint beim Buch	☑ OK
Dark Mode umschalten	Design wechselt sofort	☑ OK
Sprache wechseln (DE/EN)	Alle Texte übersetzen sich	☑ OK
Passwort ändern	Neues Passwort funktioniert beim Login	☑ OK

6.2 API Tests mit Postman

Die Backend-Endpunkte können mit Postman getestet werden:

```
POST http://localhost:8080/auth/login
Body: { "username": "admin", "password": "admin" }

GET http://localhost:8080/books
Header: Authorization: Bearer {token}

POST http://localhost:8080/books
Header: Authorization: Bearer {token}
Body: { "title": "Test", "author": "Max", "category": "Roman" }
```

7. Reflexion

7.1 Was lief gut

- Die JWT-Authentifizierung wurde erfolgreich implementiert und schützt alle Endpunkte
- Das Frontend-Design ist modern, responsiv und benutzerfreundlich
- Alle geforderten CRUD-Operationen funktionieren zuverlässig
- Die Zusatzfunktionen (Favoriten, Notizen, Abgabedatum) erhöhen den Nutzen der App
- Dark/Light Mode und Mehrsprachigkeit verbessern die Benutzererfahrung

7.2 Herausforderungen

- CORS-Konfiguration: Die Verbindung zwischen Frontend (Port 5173) und Backend (Port 8080) erforderte spezifische CORS-Einstellungen in Spring Security
- PDF-Export: Externe Libraries wie jsPDF mussten via CDN eingebunden werden, da npm-basierte Lösungen Installationsprobleme verursachten
- JSX-Struktur: Bei grossen React-Komponenten können verschachtelte Tags zu schwer findbaren Syntaxfehlern führen

7.3 Fazit

Das Projekt hat die Anforderungen des Moduls M295 vollständig erfüllt. Das Backend wurde mit Spring Boot und JWT-Sicherheit realisiert, das Frontend mit React als moderne Single Page Application. Alle CRUD-Operationen, die Authentifizierung sowie zahlreiche Zusatzfunktionen wurden erfolgreich implementiert und getestet. Die Applikation ist funktionsfähig und präsentationsbereit.

8. Anhang

8.1 Quellen & Hilfsmittel

- Spring Boot Dokumentation: https://spring.io/projects/spring-boot
- React Dokumentation: https://react.dev
- JWT Standard: https://jwt.io
- jsPDF CDN: https://cdnjs.cloudflare.com
- Vite Build Tool: https://vitejs.dev

8.2 Erklärung

Ich erkläre hiermit, dass ich diese Projektdokumentation selbstständig verfasst und alle verwendeten Hilfsmittel und Quellen angegeben habe.

Indujan Vijayarajah	März 2026
Unterschrift	Datum